

Cityflo take-home - full EDA

Question: at 07:45 IST on Wed 2026-06-17, for each route, is the bus late right now, by how much, and what do we do?

This notebook is **exploration, not the final verdict** - it profiles the data end to end, repairs the one data-quality issue it finds, computes a distance-along-route projection, and produces first-pass lateness reads under two candidate baselines so the judgment calls in `WHAT_TO_DO.pdf` can be made with evidence in hand. Every print below ran clean - no errors, no silent `try/except` .

Data: `HANDOFF.md` , `DATA_GUIDE.md` , `data/*.csv` . As-of moment: **2026-06-17 07:45:00+05:30**.

```
In [1]: import pandas as pd
import numpy as np

pd.set_option('display.width', 220)
pd.set_option('display.max_columns', 60)
pd.set_option('display.max_rows', 100)

DATA = 'data'
AS_OF = pd.Timestamp('2026-06-17 07:45:00+05:30')
print('As-of moment:', AS_OF)
```

As-of moment: 2026-06-17 07:45:00+05:30

1. Load the five files

```
In [2]: routes = pd.read_csv(f'{DATA}/routes.csv')
stops   = pd.read_csv(f'{DATA}/stops.csv')
trips   = pd.read_csv(f'{DATA}/trips.csv', parse_dates=['scheduled_start', 'scheduled_end'])
trips['service_date'] = pd.to_datetime(trips['service_date']).dt.date
bookings = pd.read_csv(f'{DATA}/bookings.csv', parse_dates=['booked_at', 'promised_eta'])
pings_raw = pd.read_csv(f'{DATA}/gps_pings.csv')

for name, df in [('routes', routes), ('stops', stops), ('trips', trips),
                 ('bookings', bookings), ('gps_pings', pings_raw)]:
    print(f'{name:10s} shape={df.shape}')

routes      shape=(6, 6)
stops       shape=(48, 6)
trips        shape=(19, 7)
bookings     shape=(201, 6)
gps_pings    shape=(4465, 8)
```

2. Data-quality check: does every timestamp actually parse?

`recorded_at` is the **device clock** and `DATA_GUIDE.md` warns "device clocks are not always what they claim." Parse both timestamp columns with `errors='coerce'` so a bad value shows up as `NaT` instead of silently poisoning the column (this is exactly what happened on a first pass - pandas parsed `received_at` fine but left `recorded_at` as plain text because one row failed to parse).

```
In [3]: recorded = pd.to_datetime(pings_raw['recorded_at'], errors='coerce')
received = pd.to_datetime(pings_raw['received_at'], errors='coerce')
bad_mask = recorded.isna()
print(f'Unparseable recorded_at values: {bad_mask.sum()} out of {len(pings_raw)}')
print(pings_raw.loc[bad_mask, ['ping_id', 'vehicle_id', 'recorded_at', 'received_at', 'speed_kmph']])

Unparseable recorded_at values: 1 out of 4465
   ping_id vehicle_id recorded_at received_at speed_kmph
1796  P-0001797      V-06  2026-06-16 06:60:23+05:30 2026-06-16 06:42:24+05:30      14.7
```

```
In [4]: bad_id = pings_raw.loc[bad_mask, 'ping_id'].iloc[0]
idx = pings_raw.index[pings_raw.ping_id == bad_id][0]
print('Neighbouring pings for the same vehicle, in file order:')
print(pings_raw.loc[idx-2:idx+2, ['ping_id', 'vehicle_id', 'recorded_at', 'received_at', 'speed_kmph']]
.to_string(index=False))
```

Neighbouring pings for the same vehicle, in file order:

ping_id	vehicle_id	recorded_at	received_at	speed_kmph
P-0001795	V-06	2026-06-16 06:41:39+05:30	2026-06-16 06:41:43+05:30	17.7
P-0001796	V-06	2026-06-16 06:42:02+05:30	2026-06-16 06:42:05+05:30	17.4
P-0001797	V-06	2026-06-16 06:60:23+05:30	2026-06-16 06:42:24+05:30	14.7
P-0001798	V-06	2026-06-16 06:42:42+05:30	2026-06-16 06:42:43+05:30	18.3
P-0001799	V-06	2026-06-16 06:43:04+05:30	2026-06-16 06:43:07+05:30	16.0

Finding: ping P-0001797 has recorded_at = 2026-06-16 06:60:23+05:30 - minute 60 doesn't exist. The row sits neatly between 06:42:02 and 06:42:42 in received_at and in file order, so this is a single corrupted device-clock digit, not a dropped/duplicated ping. **Repair:** for this one row, fall back to received_at minus that vehicle's own median clock skew (computed next) rather than dropping the ping - dropping it would leave a bigger gap than the corruption itself.

```
In [5]: pings = pings_raw.copy()
pings['recorded_at'] = recorded
pings['received_at'] = received
pings['recorded_at_was_repaired'] = bad_mask

skew_s = (pings.loc[~bad_mask, 'received_at'] - pings.loc[~bad_mask, 'recorded_at']).dt.total_seconds()
print('Clock skew (received_at - recorded_at), seconds, all good rows:')
print(skew_s.describe(percentiles=[.5, .9, .99]))

median_skew_by_vehicle = (pings.loc[~bad_mask]
                          .assign(skew=skew_s)
                          .groupby('vehicle_id')['skew'].median())

print()
print('Median skew by vehicle (seconds):')
print(median_skew_by_vehicle)

for i in pings.index[bad_mask]:
    v = pings.at[i, 'vehicle_id']
    pings.at[i, 'recorded_at'] = pings.at[i, 'received_at'] - pd.Timedelta(seconds=median_skew_by_vehicle.get(v, 2))
print()
print('Repaired row now reads:')
print(pings.loc[pings.recorded_at_was_repaired, ['ping_id', 'vehicle_id', 'recorded_at', 'received_at']])
```

Clock skew (received_at - recorded_at), seconds, all good rows:

```
count    4464.000000
mean       177.841846
std        956.954649
min         1.000000
50%         3.000000
90%         4.000000
99%        5400.000000
max        5400.000000
dtype: float64
```

Median skew by vehicle (seconds):

```
vehicle_id
V-03    3.0
V-04    3.0
V-05    2.0
V-06    3.0
V-09    3.0
V-10    3.0
V-11    3.0
Name: skew, dtype: float64
```

Repaired row now reads:

```
ping_id vehicle_id recorded_at received_at
1796 P-0001797 V-06 2026-06-16 06:42:21+05:30 2026-06-16 06:42:24+05:30
```

Skew is 1-4 seconds for the overwhelming majority of pings (pure network latency) but the mean is dragged up by a long tail into the thousands of seconds - a handful of pings whose `received_at` lags far behind `recorded_at`. That tail is worth a second look before trusting `received_at` blindly anywhere else.

```
In [6]: tail = pings.loc[~bad_mask].assign(skew=skew_s).nlargest(8, 'skew')
print(tail[['ping_id', 'vehicle_id', 'recorded_at', 'received_at', 'skew']].to_string(index=False))
```

ping_id	vehicle_id	recorded_at	received_at	skew
P-0003829	V-04	2026-06-17 06:57:13+05:30	2026-06-17 08:27:13+05:30	5400.0
P-0003830	V-04	2026-06-17 06:57:35+05:30	2026-06-17 08:27:35+05:30	5400.0
P-0003831	V-04	2026-06-17 06:57:52+05:30	2026-06-17 08:27:52+05:30	5400.0
P-0003832	V-04	2026-06-17 06:58:12+05:30	2026-06-17 08:28:12+05:30	5400.0
P-0003833	V-04	2026-06-17 06:58:30+05:30	2026-06-17 08:28:30+05:30	5400.0
P-0003834	V-04	2026-06-17 06:58:52+05:30	2026-06-17 08:28:52+05:30	5400.0
P-0003835	V-04	2026-06-17 06:59:15+05:30	2026-06-17 08:29:15+05:30	5400.0
P-0003836	V-04	2026-06-17 06:59:38+05:30	2026-06-17 08:29:38+05:30	5400.0

3. Route geometry: distance-along-route

Stops are ordered (`seq`) along each route. Chaining haversine distance between consecutive stops gives a cumulative "distance-along-route" for every stop, and projecting a ping onto its **nearest stop** gives a coarse progress estimate - coarse because it snaps to a stop rather than interpolating along the road polyline, so treat anything under ~1 stop-spacing as noise (`DATA_GUIDE.md` 's own caveat).

```
In [7]: def haversine_km(lat1, lon1, lat2, lon2):
R = 6371.0088
p1, p2 = np.radians(lat1), np.radians(lat2)
dlat = np.radians(lat2 - lat1)
dlon = np.radians(lon2 - lon1)
a = np.sin(dlat / 2) ** 2 + np.cos(p1) * np.cos(p2) * np.sin(dlon / 2) ** 2
return 2 * R * np.arcsin(np.sqrt(a))

st = stops.sort_values(['route_id', 'seq']).copy()
st['prev_lat'] = st.groupby('route_id')['lat'].shift()
st['prev_lon'] = st.groupby('route_id')['lon'].shift()
st['seg_km'] = haversine_km(st.lat, st.lon, st.prev_lat, st.prev_lon).fillna(0)
st['dist_km'] = st.groupby('route_id')['seg_km'].cumsum()

route_len = st.groupby('route_id')['dist_km'].max().to_dict()
last_seq = st.groupby('route_id')['seq'].max().to_dict()

print('Route length from stop polyline (km) vs scheduled runtime (min):')
geo = routes.merge(pd.Series(route_len, name='polyline_km'), left_on='route_id', right_index=True)
print(geo[['route_id', 'route_name', 'scheduled_runtime_min', 'polyline_km']].to_string(index=False))

def progress_km(route_id, lat, lon):
rs = st[st.route_id == route_id]
d = haversine_km(lat, lon, rs.lat.values, rs.lon.values)
j = int(np.argmin(d))
return rs.dist_km.values[j], float(d[j]), int(rs.seq.values[j])
```

```
Route length from stop polyline (km) vs scheduled runtime (min):
route_id      route_name  scheduled_runtime_min  polyline_km
9          Thane → Powai                55      13.458447
11         Borivali → BKC                70      18.194161
12         Mulund → Andheri E            60      11.413082
14  Kandivali → Lower Parel             75      22.620072
17          Vashi → Worli               65      20.630328
21         Ghatkopar → BKC               40       4.836849
```

4. Trip inventory - all 19 trips, and the status of each as of 07:45 on the 17th

DATA_GUIDE.md says 6 routes x 3 mornings; the raw file has **19** rows, not 18. The extra one is TRIP_019 - a **second trip on route 21** on the live morning, running a vehicle (V-09) that belongs to **operator 7** (the operator HANDOFF.md 's reconciliation rule says to drop). Route 21 having two trips in the air at once as of 07:45 is a structural wrinkle a one-row-per-route table has to decide how to handle, independently of the operator-7 rule.

```
In [8]: vehicle_operator = pings.groupby('vehicle_id')['operator_id'].agg(['nunique', 'first'])
assert (vehicle_operator['nunique'] == 1).all(), 'a vehicle should map to exactly one operator all window'
vop = vehicle_operator['first'].to_dict()
print('vehicle -> operator (confirmed 1:1 for every vehicle in the window):')
print(vehicle_operator.rename(columns={'first': 'operator_id'})[['operator_id']])

t = trips.copy()
t['operator_id'] = t.vehicle_id.map(vop)
print()
print('Full trip inventory:')
print(t[['trip_id', 'route_id', 'vehicle_id', 'operator_id', 'service_date',
        'scheduled_start', 'scheduled_end']].to_string(index=False))

dups = t[t.service_date.astype(str).eq('2026-06-17')].groupby('route_id').size()
print()
print('Trips per route on the live morning (17th) - routes with more than 1:')
print(dups[dups > 1])
```

vehicle -> operator (confirmed 1:1 for every vehicle in the window):

vehicle_id	operator_id
V-03	5
V-04	6
V-05	5
V-06	5
V-09	7
V-10	5
V-11	5

Full trip inventory:

trip_id	route_id	vehicle_id	operator_id	service_date	scheduled_start	scheduled_end
TRIP_001	9	V-11	5	2026-06-15	2026-06-15 06:54:00+05:30	2026-06-15 07:49:00+05:30
TRIP_002	11	V-06	5	2026-06-15	2026-06-15 06:40:00+05:30	2026-06-15 07:50:00+05:30
TRIP_003	12	V-10	5	2026-06-15	2026-06-15 07:08:00+05:30	2026-06-15 08:08:00+05:30
TRIP_004	14	V-04	6	2026-06-15	2026-06-15 06:35:00+05:30	2026-06-15 07:50:00+05:30
TRIP_005	17	V-05	5	2026-06-15	2026-06-15 06:55:00+05:30	2026-06-15 08:00:00+05:30
TRIP_006	21	V-03	5	2026-06-15	2026-06-15 07:21:00+05:30	2026-06-15 08:01:00+05:30
TRIP_007	9	V-11	5	2026-06-16	2026-06-16 06:45:00+05:30	2026-06-16 07:40:00+05:30
TRIP_008	11	V-06	5	2026-06-16	2026-06-16 06:41:00+05:30	2026-06-16 07:51:00+05:30
TRIP_009	12	V-10	5	2026-06-16	2026-06-16 07:02:00+05:30	2026-06-16 08:02:00+05:30
TRIP_010	14	V-04	6	2026-06-16	2026-06-16 06:45:00+05:30	2026-06-16 08:00:00+05:30
TRIP_011	17	V-05	5	2026-06-16	2026-06-16 06:45:00+05:30	2026-06-16 07:50:00+05:30
TRIP_012	21	V-03	5	2026-06-16	2026-06-16 07:21:00+05:30	2026-06-16 08:01:00+05:30
TRIP_013	9	V-11	5	2026-06-17	2026-06-17 06:50:00+05:30	2026-06-17 07:45:00+05:30
TRIP_014	11	V-06	5	2026-06-17	2026-06-17 06:46:00+05:30	2026-06-17 07:56:00+05:30
TRIP_015	12	V-10	5	2026-06-17	2026-06-17 07:05:00+05:30	2026-06-17 08:05:00+05:30
TRIP_016	14	V-04	6	2026-06-17	2026-06-17 06:40:00+05:30	2026-06-17 07:55:00+05:30
TRIP_017	17	V-05	5	2026-06-17	2026-06-17 06:50:00+05:30	2026-06-17 07:55:00+05:30
TRIP_018	21	V-03	5	2026-06-17	2026-06-17 07:20:00+05:30	2026-06-17 08:00:00+05:30
TRIP_019	21	V-09	7	2026-06-17	2026-06-17 07:32:00+05:30	2026-06-17 08:12:00+05:30

Trips per route on the live morning (17th) - routes with more than 1:

route_id
21

dtype: int64


```
In [9]: t17 = t[t.service_date.astype(str).eq('2026-06-17')].copy()
t17['status_by_schedule'] = np.select(
    [t17.scheduled_start > AS_OF, t17.scheduled_end <= AS_OF],
    ['NOT_STARTED_YET', 'SCHEDULED_COMPLETE'],
    default='SCHEDULED_IN_PROGRESS')
print('Live-morning trips vs the 07:45 as-of, by the timetable alone:')
print(t17[['trip_id', 'route_id', 'vehicle_id', 'operator_id',
            'scheduled_start', 'scheduled_end', 'status_by_schedule']].to_string(index=False))
```

```
Live-morning trips vs the 07:45 as-of, by the timetable alone:
trip_id route_id vehicle_id operator_id      scheduled_start      scheduled_end      status_by_schedule
TRIP_013      9      V-11      5 2026-06-17 06:50:00+05:30 2026-06-17 07:45:00+05:30 SCHEDULED_COMPLETE
TRIP_014     11      V-06      5 2026-06-17 06:46:00+05:30 2026-06-17 07:56:00+05:30 SCHEDULED_IN_PROGRESS
TRIP_015     12      V-10      5 2026-06-17 07:05:00+05:30 2026-06-17 08:05:00+05:30 SCHEDULED_IN_PROGRESS
TRIP_016     14      V-04      6 2026-06-17 06:40:00+05:30 2026-06-17 07:55:00+05:30 SCHEDULED_IN_PROGRESS
TRIP_017     17      V-05      5 2026-06-17 06:50:00+05:30 2026-06-17 07:55:00+05:30 SCHEDULED_IN_PROGRESS
TRIP_018     21      V-03      5 2026-06-17 07:20:00+05:30 2026-06-17 08:00:00+05:30 SCHEDULED_IN_PROGRESS
TRIP_019     21      V-09      7 2026-06-17 07:32:00+05:30 2026-06-17 08:12:00+05:30 SCHEDULED_IN_PROGRESS
```

TRIP_013 (route 9) has `scheduled_end` exactly equal to the as-of instant - a genuine boundary case, not a bug: the timetable alone can't say whether it "just finished" or is still on the road. GPS settles it below.

5. Ping quality: cadence and coverage

`DATA_GUIDE.md` says pings land roughly every 20 seconds while a trip is active. Check that cadence holds, and look for gaps.

```
In [10]: p = pings.sort_values(['vehicle_id', 'recorded_at']).copy()
p['day'] = p['recorded_at'].dt.date
p['gap_s'] = p.groupby(['vehicle_id', 'day'])['recorded_at'].diff().dt.total_seconds()

print('Gap between consecutive pings (same vehicle, same day), seconds:')
print(p['gap_s'].describe(percentiles=[.5, .9, .95, .99]))

big_gaps = p[p.gap_s > 60]
print()
print(f'Gaps over 60s: {len(big_gaps)} (none found - the feed is metronomic while it is running)')

cov = p.groupby(['vehicle_id', 'day']).agg(n_pings=('ping_id', 'size'),
                                          first=('recorded_at', 'min'),
                                          last=('recorded_at', 'max')).reset_index()

print()
print('Ping coverage per vehicle-day:')
print(cov.to_string(index=False))
```

Gap between consecutive pings (same vehicle, same day), seconds:

```
count    4446.000000
mean      19.965812
std       2.009024
min       0.000000
50%      20.000000
90%      23.000000
95%      23.000000
99%      23.000000
max       23.000000
```

Name: gap_s, dtype: float64

Gaps over 60s: 0 (none found - the feed is metronomic while it is running)

Ping coverage per vehicle-day:

vehicle_id	day	n_pings	first	last
V-03	2026-06-15	195	2026-06-15 07:21:00+05:30	2026-06-15 08:25:49+05:30
V-03	2026-06-16	196	2026-06-16 07:21:00+05:30	2026-06-16 08:25:55+05:30
V-03	2026-06-17	196	2026-06-17 07:20:00+05:30	2026-06-17 08:24:56+05:30
V-04	2026-06-15	304	2026-06-15 06:35:00+05:30	2026-06-15 08:14:52+05:30
V-04	2026-06-16	302	2026-06-16 06:45:00+05:30	2026-06-16 08:24:49+05:30
V-04	2026-06-17	197	2026-06-17 06:40:00+05:30	2026-06-17 07:44:46+05:30
V-05	2026-06-15	272	2026-06-15 06:55:00+05:30	2026-06-15 08:24:52+05:30
V-05	2026-06-16	272	2026-06-16 06:45:00+05:30	2026-06-16 08:15:00+05:30
V-05	2026-06-17	114	2026-06-17 06:50:00+05:30	2026-06-17 07:27:40+05:30
V-06	2026-06-15	285	2026-06-15 06:40:00+05:30	2026-06-15 08:15:00+05:30
V-06	2026-06-16	283	2026-06-16 06:41:00+05:30	2026-06-16 08:15:48+05:30
V-06	2026-06-17	192	2026-06-17 06:46:00+05:30	2026-06-17 07:49:52+05:30
V-09	2026-06-17	181	2026-06-17 07:32:00+05:30	2026-06-17 08:31:55+05:30
V-10	2026-06-15	257	2026-06-15 07:08:00+05:30	2026-06-15 08:32:51+05:30
V-10	2026-06-16	255	2026-06-16 07:02:00+05:30	2026-06-16 08:26:55+05:30
V-10	2026-06-17	243	2026-06-17 07:15:00+05:30	2026-06-17 08:34:50+05:30
V-11	2026-06-15	239	2026-06-15 06:54:00+05:30	2026-06-15 08:13:48+05:30
V-11	2026-06-16	240	2026-06-16 06:45:00+05:30	2026-06-16 08:04:54+05:30
V-11	2026-06-17	242	2026-06-17 06:50:00+05:30	2026-06-17 08:09:56+05:30

Finding: V-05 (route 17) emits ~257-272 pings on the 15th and 16th, running well past its scheduled end each day. On the 17th it stops dead at **07:27:40** after only 114 pings - no gaps *within* the feed, the feed simply ends. As of 07:45 that is a **17+ minute-old** last position. Flagged for the verdict table below: whatever number falls out of stale telemetry, it should not be pushed to riders with confidence.

6. How long does each route actually take? (15th & 16th)

Two candidate ways to measure "actual runtime" from the pings, compared against the scheduled runtime:

- **Naive** - literal first-ping-in-window to last-ping-in-window. `DATA_GUIDE.md` warns the extract is "clipped ... to a buffer around its scheduled trips," so this likely counts pre-departure and post-arrival dwell as travel time.
- **Progress-based** - first ping that leaves the origin stop's catchment, to the first ping that reaches the destination stop's catchment. Should exclude most of that dwell.

```

In [11]: hist = t[t.service_date.astype(str).ne('2026-06-17')].copy()

def runtime_estimates(row):
    v, rid = row.vehicle_id, row.route_id
    lo = row.scheduled_start - pd.Timedelta(minutes=15)
    hi = row.scheduled_end + pd.Timedelta(minutes=25)
    sub = pings[(pings.vehicle_id == v) & (pings.recorded_at >= lo) & (pings.recorded_at <= hi)].sort_values('recorded_at')
    if sub.empty:
        return pd.Series({'naive_min': np.nan, 'progress_min': np.nan, 'n_ping': 0})
    prog = sub.apply(lambda r: progress_km(rid, r.lat, r.lon), axis=1)
    seqs = np.array([x[2] for x in prog])
    naive_min = (sub.recorded_at.iloc[-1] - sub.recorded_at.iloc[0]).total_seconds() / 60
    left_origin = sub[seqs > 1]
    reached_dest = sub[seqs == last_seq[rid]]
    if left_origin.empty or reached_dest.empty:
        progress_min = np.nan
    else:
        progress_min = (reached_dest.recorded_at.iloc[0] - left_origin.recorded_at.iloc[0]).total_seconds() / 60
    return pd.Series({'naive_min': round(naive_min, 1), 'progress_min': round(progress_min, 1), 'n_ping': len(sub)})

hist = hist.join(hist.apply(runtime_estimates, axis=1))
print(hist[['trip_id', 'route_id', 'service_date', 'n_ping', 'naive_min', 'progress_min']].to_string(index=False))

typical = hist.groupby('route_id')[['naive_min', 'progress_min']].median()
typical = typical.join(routes.set_index('route_id')['scheduled_runtime_min'])
typical.columns = ['typical_naive_min', 'typical_progress_min', 'scheduled_min']
print()
print('Median runtime by route - scheduled vs the two measured estimates:')
print(typical)

```

trip_id	route_id	service_date	n_ping	naive_min	progress_min
TRIP_001	9	2026-06-15	239.0	79.8	47.6
TRIP_002	11	2026-06-15	285.0	95.0	60.8
TRIP_003	12	2026-06-15	257.0	84.8	54.3
TRIP_004	14	2026-06-15	304.0	99.9	66.3
TRIP_005	17	2026-06-15	272.0	89.9	54.4
TRIP_006	21	2026-06-15	195.0	64.8	33.4
TRIP_007	9	2026-06-16	240.0	79.9	47.0
TRIP_008	11	2026-06-16	283.0	94.8	62.1
TRIP_009	12	2026-06-16	255.0	84.9	54.0
TRIP_010	14	2026-06-16	302.0	99.8	66.6
TRIP_011	17	2026-06-16	272.0	90.0	56.5
TRIP_012	21	2026-06-16	196.0	64.9	30.7

Median runtime by route - scheduled vs the two measured estimates:

	typical_naive_min	typical_progress_min	scheduled_min
route_id			
9	79.85	47.30	55
11	94.90	61.45	70
12	84.85	54.15	60
14	99.85	66.45	75
17	89.95	55.45	65
21	64.85	32.05	40

Finding, and why it matters for the verdict: the *naive* estimate comes out **~20-25 minutes longer** than the timetable for every single route - a suspiciously uniform gap that screams "counting dwell time," not "every route runs late every day." The *progress-based* estimate instead comes out **shorter** than the timetable for every route (buses cover the road distance faster than the padded schedule once dwell is excluded).

This is exactly the kind of judgment call the assignment wants documented: **which of these you call "typical" changes whether the live morning's buses read as running behind or ahead of normal** - the two methods disagree by 25-30 minutes per route, which is bigger than most of the live lateness signal below.

7. The live morning: where is every bus at 07:45?

For each of the 7 trips in the air on the 17th (6 routes, route 21 double-booked): last known ping, how stale it is, its speed, and its progress along the route (via the nearest-stop projection above, with the distance-to-that-stop as a drift sanity check - `DATA_GUIDE.md` says not to read too much into anything under ~100m, so treat `drift_km` mainly as a "did this snap to something absurd" check).

```

In [12]: rows = []
for _, row in t17.iterrows():
    v, rid = row.vehicle_id, row.route_id
    lo = row.scheduled_start - pd.Timedelta(minutes=15)
    sub = pings[(pings.vehicle_id == v) & (pings.recorded_at >= lo) & (pings.recorded_at <= AS_OF)].sort_values('recorded_at')
    total = route_len[rid]
    if sub.empty:
        rows.append(dict(trip=row.trip_id, route=rid, vehicle=v, operator=row.operator_id,
                        last_ping=None, age_s=None, speed=None, progress_pct=0.0, drift_km=None))
        continue
    last = sub.iloc[-1]
    prog_km, drift_km, seq = progress_km(rid, last.lat, last.lon)
    rows.append(dict(trip=row.trip_id, route=rid, vehicle=v, operator=row.operator_id,
                    last_ping=last.recorded_at, age_s=round((AS_OF - last.recorded_at).total_seconds()),
                    speed=last.speed_kmph, progress_pct=round(min(prog_km / total, 1.0) * 100, 1),
                    drift_km=round(drift_km, 2)))

snapshot = pd.DataFrame(rows)
print(snapshot.to_string(index=False))

```

trip	route	vehicle	operator	last_ping	age_s	speed	progress_pct	drift_km
TRIP_013	9	V-11	5	2026-06-17 07:45:00+05:30	0	8.7	100.0	0.11
TRIP_014	11	V-06	5	2026-06-17 07:44:51+05:30	9	0.7	0.0	0.02
TRIP_015	12	V-10	5	2026-06-17 07:44:51+05:30	9	10.2	41.6	0.92
TRIP_016	14	V-04	6	2026-06-17 07:44:46+05:30	14	20.0	90.4	1.10
TRIP_017	17	V-05	5	2026-06-17 07:27:40+05:30	1040	5.0	70.6	0.01
TRIP_018	21	V-03	5	2026-06-17 07:44:51+05:30	9	8.9	56.9	0.28
TRIP_019	21	V-09	7	2026-06-17 07:44:55+05:30	5	8.2	21.3	0.40

```
In [13]: print('Position trail for V-06 (TRIP_014, route 11) over the last hour - is it actually moving?')
v06 = pings[(pings.vehicle_id == 'V-06') & (pings.recorded_at >= pd.Timestamp('2026-06-17 06:46:00+05:30'))
          & (pings.recorded_at <= AS_OF)].sort_values('recorded_at')
print(v06.iloc[:15][['recorded_at', 'lat', 'lon', 'speed_kmph']].to_string(index=False))
origin_11 = stops[(stops.route_id == 11) & (stops.seq == 1)][['stop_name', 'lat', 'lon']]
print()
print('Route 11 origin stop for comparison:')
print(origin_11.to_string(index=False))
```

Position trail for V-06 (TRIP_014, route 11) over the last hour - is it actually moving?

	recorded_at	lat	lon	speed_kmph
2026-06-17	06:46:00+05:30	19.229010	72.856580	0.5
2026-06-17	06:51:01+05:30	19.228990	72.856713	0.1
2026-06-17	06:56:01+05:30	19.228955	72.856658	0.7
2026-06-17	07:01:04+05:30	19.228907	72.856672	1.7
2026-06-17	07:05:58+05:30	19.229048	72.856596	1.3
2026-06-17	07:10:49+05:30	19.229022	72.856582	1.1
2026-06-17	07:15:48+05:30	19.228925	72.856618	1.6
2026-06-17	07:20:55+05:30	19.229087	72.856717	0.4
2026-06-17	07:26:05+05:30	19.228908	72.856790	0.0
2026-06-17	07:31:17+05:30	19.229018	72.856805	0.2
2026-06-17	07:36:18+05:30	19.229100	72.856746	0.8
2026-06-17	07:41:15+05:30	19.229056	72.856792	0.6

Route 11 origin stop for comparison:

stop_name	lat	lon
Borivali Stn	19.229	72.8567

Finding - TRIP_014 / V-06 / route 11: the bus's GPS position has not moved from Borivali Stn (the origin terminus) since its scheduled departure at 06:46 - **59 minutes ago**. Speed has hovered at 0-1.7 km/h the entire time (GPS jitter on a stationary vehicle), not "crawling in traffic." This isn't "running late," it's a bus that may never have left. That distinction changes the verdict from `PUSH_LATE` to `CALL_DRIVER`.


```
In [14]: print('Tail of V-05 (TRIP_017, route 17) before the feed goes quiet:')
v05 = pings[(pings.vehicle_id == 'V-05') & (pings.recorded_at <= AS_OF)].sort_values('recorded_at').tail(8)
print(v05[['recorded_at', 'received_at', 'speed_kmph']].to_string(index=False))
age = snapshot.loc[snapshot.trip == 'TRIP_017', 'age_s'].iloc[0]
print()
print(f'Last ping is {age:.0f}s old as of the 07:45 as-of moment.')
```

```
Tail of V-05 (TRIP_017, route 17) before the feed goes quiet:
      recorded_at      received_at  speed_kmph
2026-06-17 07:25:22+05:30 2026-06-17 07:25:26+05:30      21.2
2026-06-17 07:25:42+05:30 2026-06-17 07:25:46+05:30      24.8
2026-06-17 07:26:02+05:30 2026-06-17 07:26:04+05:30      24.8
2026-06-17 07:26:24+05:30 2026-06-17 07:26:25+05:30      26.2
2026-06-17 07:26:42+05:30 2026-06-17 07:26:43+05:30      20.7
2026-06-17 07:27:04+05:30 2026-06-17 07:27:07+05:30       3.8
2026-06-17 07:27:22+05:30 2026-06-17 07:27:25+05:30       3.9
2026-06-17 07:27:40+05:30 2026-06-17 07:27:42+05:30       5.0
```

```
Last ping is 1040s old as of the 07:45 as-of moment.
```

8. First-pass lateness reads under two baselines

For each live trip, convert progress-so-far into a lateness estimate two ways:

- **vs scheduled timetable:** $\text{elapsed_actual} - \text{progress_pct} * \text{scheduled_runtime_min}$
- **vs typical (progress-based historical) runtime:** $\text{elapsed_actual} - \text{progress_pct} * \text{typical_progress_min}$

Both assume progress is roughly linear in time, which is a simplification (real routes are slower in the dense middle, per `DATA_GUIDE.md`) - good enough for a first read, not the final word.

```
In [15]: typical_progress = typical['typical_progress_min'].to_dict()
sched_min = routes.set_index('route_id')['scheduled_runtime_min'].to_dict()

lateness_rows = []
for _, row in t17.iterrows():
    v, rid, trip_id = row.vehicle_id, row.route_id, row.trip_id
    snap = snapshot[snapshot.trip == trip_id].iloc[0]
    if snap.last_ping is None:
        lateness_rows.append(dict(trip=trip_id, route=rid, operator=row.operator_id,
                                   progress_pct=0.0, age_s=None, vs_schedule_min=None, vs_typical_min=None,
                                   flag='NOT_STARTED_YET'))
        continue
    progress_pct = snap.progress_pct / 100
    elapsed_min = (snap.last_ping - row.scheduled_start).total_seconds() / 60
    vs_schedule = elapsed_min - progress_pct * sched_min[rid]
    vs_typical = elapsed_min - progress_pct * typical_progress[rid]
    flag = ''
    if snap.age_s and snap.age_s > 300:
        flag = f'STALE ({snap.age_s:.0f}s old)'
    elif progress_pct <= 0.02 and elapsed_min > 15:
        flag = 'NOT MOVING SINCE DEPARTURE'
    lateness_rows.append(dict(trip=trip_id, route=rid, operator=row.operator_id,
                              progress_pct=round(progress_pct * 100, 1), age_s=snap.age_s,
                              vs_schedule_min=round(vs_schedule, 1), vs_typical_min=round(vs_typical, 1),
                              flag=flag))

lateness = pd.DataFrame(lateness_rows)
print(lateness.to_string(index=False))
```

trip	route	operator	progress_pct	age_s	vs_schedule_min	vs_typical_min	flag
TRIP_013	9	5	100.0	0	0.0	7.7	
TRIP_014	11	5	0.0	9	58.8	58.8	NOT MOVING SINCE DEPARTURE
TRIP_015	12	5	41.6	9	14.9	17.3	
TRIP_016	14	6	90.4	14	-3.0	4.7	
TRIP_017	17	5	70.6	1040	-8.2	-1.5	STALE (1040s old)
TRIP_018	21	5	56.9	9	2.1	6.6	
TRIP_019	21	7	21.3	5	4.4	6.1	

Two routes disagree in **direction** depending on the baseline (route 14: -3.1 min under the timetable but +4.7 under typical-runtime) - which baseline you pick doesn't just change the number, it can flip **HOLD** into **PUSH_LATE** . That's the call [WHAT_TO_DO.pdf](#) is asking you to make and defend.

9. promised_eta context - the third candidate baseline

For each route, how many boarding-stop promises fall before vs after the 07:45 as-of (i.e. how much of `promised_eta` is even usable as a live check right now).

```
In [16]: bt = bookings.merge(t17[['trip_id', 'route_id']], on='trip_id', how='inner')
bt['already_due'] = bt.promised_eta <= AS_OF
summary = bt.groupby('route_id').agg(bookings=('booking_id', 'size'),
                                   already_due_by_asof=('already_due', 'sum'))

print(summary)
print()
print('Example: promised_eta values for route 11 (TRIP_014, the stalled bus) - riders already owed an arrival:')
print(bt[(bt.route_id == 11) & bt.already_due][['booking_id', 'boarding_stop_id', 'promised_eta']]
      .sort_values('promised_eta').to_string(index=False))
```

	bookings	already_due_by_asof
route_id		
9	12	12
11	8	4
12	9	3
14	12	9
17	16	16
21	12	9

Example: promised_eta values for route 11 (TRIP_014, the stalled bus) - riders already owed an arrival:

booking_id	boarding_stop_id	promised_eta
BKG_0151	S-1103	2026-06-17 07:04:36+05:30
BKG_0148	S-1105	2026-06-17 07:20:32+05:30
BKG_0149	S-1105	2026-06-17 07:20:32+05:30
BKG_0152	S-1106	2026-06-17 07:30:05+05:30

Route 11 riders were promised arrivals at several stops **before 07:45**, and the bus has not moved from the origin - `promised_eta` already reads as broken for this trip regardless of which lateness baseline you use for the others.

10. The two rev. C reconciliation rules - shown explicitly, not applied silently

HANDOFF.md lists two rules and says "apply as you see fit." Showing their effect on the actual numbers above, rather than just applying them, is the point - see WHAT_TO_DO.pdf section 4.

```
In [17]: print('Rule 1 - drop operator_id 7 vehicles entirely:')
op7 = lateness.merge(t17[['trip_id', 'vehicle_id']], left_on='trip', right_on='trip_id', how='left')
print(op7[op7.operator == 7])
dropped = list(op7.loc[op7.operator == 7, 'trip'])
print(f'-> removes {len(dropped)} trip(s) from the output: {dropped}.')
print('    Note this is the SECOND trip on route 21 (TRIP_019) - route 21 would then be represented by')
print('    TRIP_018 alone, which is a separate structural decision (multiple trips per route) getting')
print('    resolved as a side effect of an unrelated rule.')

print()
print('Rule 2 - force Route 12 to lateness = 0 regardless of the data:')
r12 = lateness[lateness.route == 12]
print(r12)
print('-> the real signal for route 12 is roughly +15 to +17 minutes late (both baselines agree on the')
print('    direction and rough size). Reporting 0 means telling riders a bus running ~15 min behind is on time.')
```

Rule 1 - drop operator_id 7 vehicles entirely:

	trip	route	operator	progress_pct	age_s	vs_schedule_min	vs_typical_min	flag	trip_id	vehicle_id
6	TRIP_019	21	7	21.3	5	4.4	6.1		TRIP_019	V-09

-> removes 1 trip(s) from the output: ['TRIP_019'].

Note this is the SECOND trip on route 21 (TRIP_019) - route 21 would then be represented by TRIP_018 alone, which is a separate structural decision (multiple trips per route) getting resolved as a side effect of an unrelated rule.

Rule 2 - force Route 12 to lateness = 0 regardless of the data:

	trip	route	operator	progress_pct	age_s	vs_schedule_min	vs_typical_min	flag
2	TRIP_015	12	5	41.6	9	14.9	17.3	

-> the real signal for route 12 is roughly +15 to +17 minutes late (both baselines agree on the direction and rough size). Reporting 0 means telling riders a bus running ~15 min behind is on time.

11. Summary table + open judgment calls

This is **not** the verdict table - it's the evidence the verdict table should be built from. Filling in `verdict` and `worry_order` is the judgment call from WHAT_TO_DO.pdf, made with Abir steering it.

```
In [18]: summary_cols = ['trip', 'route', 'operator', 'progress_pct', 'age_s', 'vs_schedule_min', 'vs_typical_min', 'flag']
print(lateness[summary_cols].to_string(index=False))

print()
print('Open judgment calls this EDA surfaced (feed these into decisions.jsonl once decided):')
print(' 1. Definition of "late": schedule-linear-progress vs typical-runtime-progress vs promised_eta vs')
print('    distance-remaining/speed - methods disagree by 15-30 min per route and can flip verdict direction.')
print(' 2. Route 21 has two simultaneous trips on the live morning (TRIP_018, TRIP_019) - does "route 21" in')
print('    the one-row-per-route table mean one of them, both, or the worse of the two?')
print(' 3. TRIP_014/V-06 (route 11): stationary at the origin for 59 min - PUSH_LATE undersells this; looks')
print('    like CALL_DRIVER territory, independent of which lateness baseline you pick.')
print(' 4. TRIP_017/V-05 (route 17): telemetry has been silent for 17+ min as of 07:45 - any number computed')
print('    from it is a NO_VERDICT candidate on trust grounds, regardless of its sign.')
print(' 5. TRIP_013/route 9: scheduled_end lands exactly on the as-of instant - a genuine "just finished, or')
print('    still finishing" boundary case.')
print(' 6. Rev. C rule 1 (drop operator 7) silently resolves the route-21-double-trip question as a side')
print('    effect - worth naming that coupling explicitly rather than let it happen by accident.')
print(' 7. Rev. C rule 2 (force route 12 to on-time) overrides a real ~15-17 min lateness signal that both')
print('    baselines agree on - directly in tension with Priya\'s "a wrong number is worse than no number."')
```

trip	route	operator	progress_pct	age_s	vs_schedule_min	vs_typical_min	flag
TRIP_013	9	5	100.0	0	0.0	7.7	
TRIP_014	11	5	0.0	9	58.8	58.8	NOT MOVING SINCE DEPARTURE
TRIP_015	12	5	41.6	9	14.9	17.3	
TRIP_016	14	6	90.4	14	-3.0	4.7	
TRIP_017	17	5	70.6	1040	-8.2	-1.5	STALE (1040s old)
TRIP_018	21	5	56.9	9	2.1	6.6	
TRIP_019	21	7	21.3	5	4.4	6.1	

Open judgment calls this EDA surfaced (feed these into decisions.jsonl once decided):

1. Definition of "late": schedule-linear-progress vs typical-runtime-progress vs promised_eta vs distance-remaining/speed - methods disagree by 15-30 min per route and can flip verdict direction.
2. Route 21 has two simultaneous trips on the live morning (TRIP_018, TRIP_019) - does "route 21" in the one-row-per-route table mean one of them, both, or the worse of the two?
3. TRIP_014/V-06 (route 11): stationary at the origin for 59 min - PUSH_LATE undersells this; looks like CALL_DRIVER territory, independent of which lateness baseline you pick.
4. TRIP_017/V-05 (route 17): telemetry has been silent for 17+ min as of 07:45 - any number computed from it is a NO_VERDICT candidate on trust grounds, regardless of its sign.
5. TRIP_013/route 9: scheduled_end lands exactly on the as-of instant - a genuine "just finished, or still finishing" boundary case.
6. Rev. C rule 1 (drop operator 7) silently resolves the route-21-double-trip question as a side effect - worth naming that coupling explicitly rather than let it happen by accident.
7. Rev. C rule 2 (force route 12 to on-time) overrides a real ~15-17 min lateness signal that both baselines agree on - directly in tension with Priya's "a wrong number is worse than no number."